Building Persistent Intelligence for AI-Assisted Research

A Knowledge Architecture for Claude Code

Larry Santucci

Federal Reserve Bank of Philadelphia

March 11, 2026

The views expressed herein are solely those of the author and do not necessarily reflect the views of the Federal Reserve Bank of Philadelphia or the Federal Reserve System. Nothing in the text should be construed as an endorsement of any organization or its products or services. This presentation is a co-creation of the author and Claude Code for VS Code version 2.1.72.

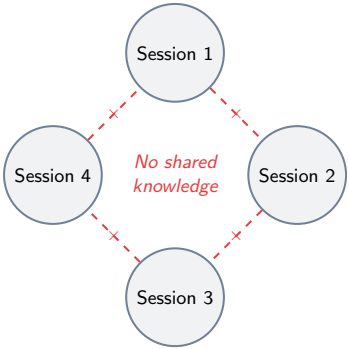
The Problem: Every Session Starts from Zero

The Default Experience

- Each Claude Code session is a blank slate
- No memory of past mistakes or fixes
- No knowledge of project conventions
- Repeats the same errors across sessions
- Asks the same questions every time

The Real Cost

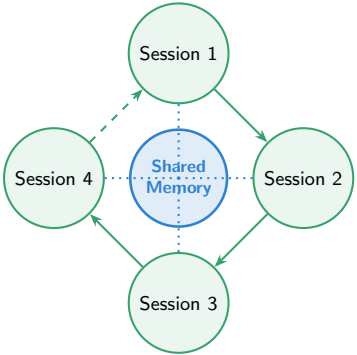
Researchers lose hours re-teaching their AI assistant things it should already know.



What If Your AI Assistant Could Learn?

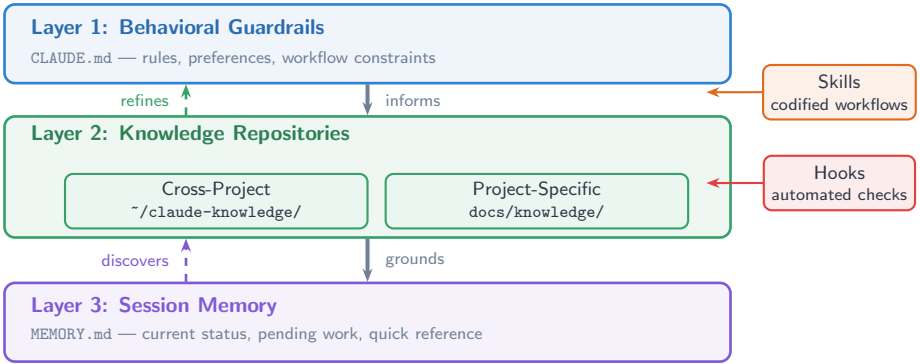
The Goal

- Remember what worked — and what didn't
- Know project conventions without being told
- Distinguish project-specific from universal knowledge
- Codify repeatable workflows automatically
- Catch known errors before they happen



Key insight: Claude Code already has the infrastructure for this. **You just have to build the architecture.**

The Three-Layer Architecture



Layer 1: Behavioral Guardrails

Two levels of `CLAUDE.md` files shape every interaction:

Global (`~/.claude/CLAUDE.md`)

- Your identity, expertise, communication style
- When to ask vs. proceed
- What frustrates you about AI assistants
- Universal rules (verify every step, no silent assumptions)
- Pointer to cross-project knowledge

Project (`project/CLAUDE.md`)

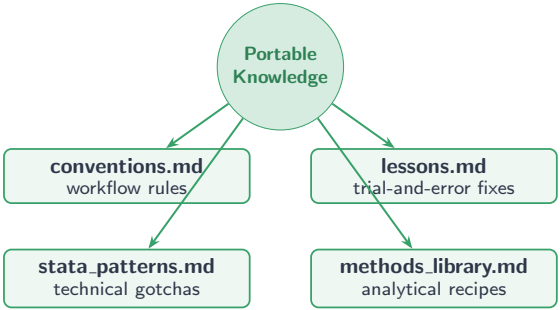
- Key variables and their meanings
- Directory structure conventions
- "Read these files first" instructions
- Knowledge management rules
- Do-file conventions

Why This Matters

Without guardrails, Claude will reorganize code you didn't ask it to touch, add unnecessary error handling, make silent assumptions about methodology, and apologize instead of asking. With guardrails, it works *the way you work*.

Layer 2a: Cross-Project Knowledge

`~/claude-knowledge/` — a git-tracked repository shared across all projects.




```
# lessons.md
csdid saverif() crashes
with directory paths.
Use bare filenames only.
[Discovered:  Mar 2026]
```

What belongs here:

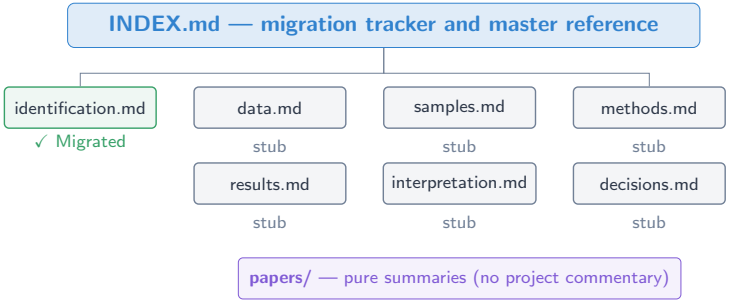
- Software bugs and workarounds
- Process conventions (verify every step)
- Analytical method recipes

What does NOT belong:

- Project-specific variable definitions
- Regression results or coefficients
- Sample construction details

Layer 2b: Project Knowledge Repository

`docs/knowledge/` — single source of truth for everything about *this* project.



Every fact lives in exactly one file. Paper notes are pure summaries — project implications go in topic files.

Layer 3: Session Memory

`MEMORY.md` — auto-loaded every session, carries forward across conversations.

Persistent (stable across sessions)

- Authors, collaborators
- File paths, software locations
- Key variable names and definitions
- User preferences and workflow rules

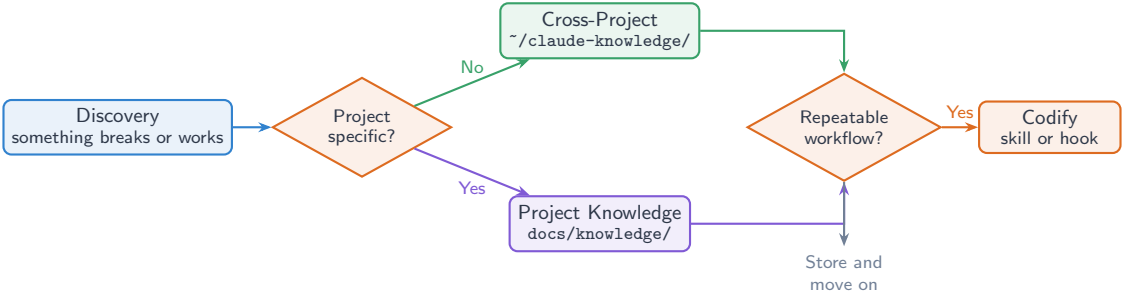
Ephemeral (verify at session start)

- Current project status
- Pending work items
- Last compilation dates
- Recently completed tasks

Design Principle

Separate what *persists* from what *decays*. Ephemeral sections are date-stamped and labeled "verify at session start" so new sessions know what might be stale.

The Knowledge Lifecycle



Retain the fix, forget the failures.

After 12 failed approaches and 1 solution, only the solution enters the knowledge base — with enough context to prevent the same failure mode.

One fact, one file.

When knowledge is discovered or corrected, update the canonical source immediately. No duplication across files.

Skills: Codified Workflows

Skills are reusable, multi-step procedures invoked with `/skill-name`.

/adapt-dofile

1. Read project context
2. Read template do-file
3. Plan changes (max 5)
4. Edit with template fidelity
5. Show diff before running

/review-paper

1. Read project context
2. Read paper (any format)
3. Extract structured notes
4. Write to papers/ directory
5. Ask about implications

/migrate-knowledge

1. Read target stub file
2. Read legacy source docs
3. Check for conflicts
4. Draft migrated content
5. Update INDEX.md

What Makes a Good Skill

- A workflow you've done 3+ times with the same structure
- A "read these files first" preamble that gets repeated
- A communication product that follows a consistent template
- Clear inputs (arguments), clear outputs (files), clear constraints (rules)

Hooks: Automated Validation

Hooks fire before or after tool use — they enforce rules without requiring manual attention.

PreToolUse: Large File Guard

Fires before `Read`. If a file exceeds 500KB, warns Claude to use `offset/limit` or convert to text first.

Prevents: accidentally dumping huge datasets into context, wasting the context window.

PostToolUse: Stata Log Validator

Fires after `Bash` when the command contains `stata-mp`. Scans output for:

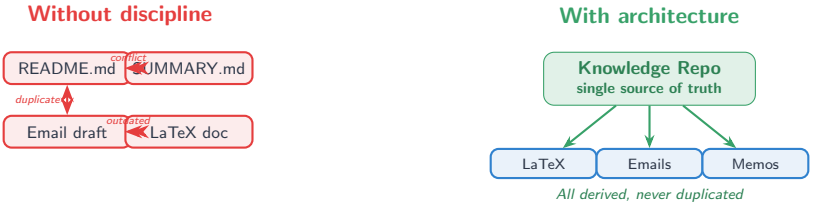
- `r(...)` error codes
- "no observations" warnings
- Memory allocation failures
- "variable not found" errors

Prevents: silently continuing past Stata errors.

When to Create a Hook

When a manual check is repeated after every run, or an error pattern has been caught and fixed more than once. Hooks belong at the *global* level (`~/.claude/settings.json`) if they apply across projects.

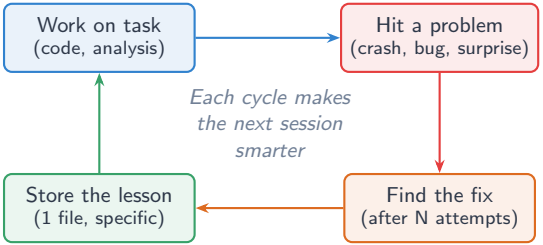
The “Single Source of Truth” Principle



The Rule

When a fact changes, you update **one file**. Everything downstream regenerates from it. If you find the same fact in two places, you have a bug in your architecture.

The Lessons Learned Loop



Example:

Problem: `csdid saverif()` crashes with directory paths

Failed: 4 path variations

Fix: bare filenames only

Stored in: `stata_patterns.md` and `lessons.md`

The key discipline: after debugging, **immediately** distill the lesson into the knowledge base. Don't promise to do it later. The 12 failed attempts get discarded; the fix gets preserved with just enough context to prevent the same failure.

Inter-Project vs. Intra-Project Knowledge

	Cross-Project `~/claude-knowledge/`	Project-Specific `docs/knowledge/`
Content	Software gotchas, process conventions, analytical recipes, debugging lessons	Variable definitions, sample construction, regression results, identification strategy
Lifespan	Indefinite — survives project completion	Tied to project lifecycle
Audience	Any future project's Claude instance	Only this project's Claude instance
Example	"`csdid` crashes on ¿2.5M obs"	"Panel A uses oversample only due to memory constraints"
Update trigger	New software discovery or workflow refinement	New results, methodology change, or course correction

The Test

If you started a completely different research project tomorrow, would this knowledge still be useful? **Yes** → cross-project. **No** → project-specific.

Building Your Own: Step 1 — Global Configuration

Time: 30 minutes. Impact: every future session.

1. **Create** `~/.claude/CLAUDE.md` — your global instruction file.
   - Who you are, your expertise level, communication preferences
   - Your frustrations with AI assistants (be specific!)
   - When Claude should ask vs. proceed autonomously
   - Universal rules ("verify every step," "no silent assumptions")
2. **Create** `~/claude-knowledge/` — your cross-project knowledge repo.
   - `conventions.md` — process rules that apply everywhere
   - `lessons.md` — debugging insights (start empty, grows organically)
   - One file per software/tool you use heavily (e.g., `stata_patterns.md`)
   - Initialize as a git repo for version tracking
3. **Configure** `~/.claude/settings.json` — permissions and hooks.
   - Set `additionalDirectories` to include your knowledge repo
   - Add a PreToolUse hook for large file warnings

Building Your Own: Step 2 — Project Knowledge

Time: 1 hour per project. Impact: eliminates re-teaching.

1. **Create** `project/CLAUDE.md` — project-specific instructions.
   - "Read these files before doing anything"
   - Key variables, directory structure, naming conventions
   - Pointer to the knowledge repo
2. **Create** `docs/knowledge/` with an `INDEX.md` and topic files.
   - Start with stubs — list what each file *will* contain
   - Migrate incrementally as you work (don't try to do it all at once)
   - One fully-written file is better than eight half-written ones
3. **Set up** `MEMORY.md` in your project memory directory.
   - Persistent section: authors, paths, key variables, preferences
   - Ephemeral section: current status, pending work (date-stamped)
   - Keep it under 200 lines (it's loaded into every context window)

Common Mistake

Don't try to document everything upfront. Start working, and when Claude asks a question you've answered before, *that's* what goes in the knowledge repo.

Building Your Own: Step 3 — Skills and Hooks

Time: ongoing. Impact: compounds over time.

When to Create a Skill

A skill is a multi-step workflow saved as a `SKILL.md` file in `.claude/skills/`.

- You've done the same workflow 3+ times
- It has a "read these files first" preamble
- It produces a consistent output format (paper notes, adapted code, migrated docs)
- It has clear rules that prevent Claude from over-engineering

When to Create a Hook

A hook is a shell command that fires before/after a tool in `settings.json`.

- A manual check is repeated after every execution
- An error pattern has burned you more than once
- A safety guard needs to be enforced without thinking about it

Skill anatomy: Name, description, arguments, 3–5 steps, rules.
Hook anatomy: Trigger (Pre/PostToolUse), matcher (tool name), shell command.

Key Principles

1. Every fact lives in exactly one place. If you find it in two files, fix that.
2. Retain the fix, forget the failures. Knowledge is distilled, not journaled.
3. Separate what persists from what decays. Date-stamp the ephemeral.
4. Cross-project knowledge is portable. Project knowledge is specific.
5. Output products are derived from the knowledge base, never duplicated.
6. Codify workflows as skills when you've done them three times.
7. Automate validation with hooks when a check is repeated after every run.

Getting Started Checklist

	Action	Location
☐	Write your global instruction file	`~/.claude/CLAUDE.md`
☐	Create cross-project knowledge repo	`~/claude-knowledge/`
☐	Add `conventions.md` with your process rules	`~/claude-knowledge/`
☐	Add `lessons.md` (start empty)	`~/claude-knowledge/`
☐	Configure permissions and directories	`~/.claude/settings.json`
☐	Add large-file warning hook	`~/.claude/settings.json`
☐	Write project-level `CLAUDE.md`	`project/CLAUDE.md`
☐	Create `docs/knowledge/INDEX.md` with topic stubs	`project/docs/knowledge/`
☐	Set up `MEMORY.md` (persistent + ephemeral)	`~/.claude/projects/.../memory/`
☐	Fully migrate your most important topic file first	`project/docs/knowledge/`
☐	Create your first skill after 3 repeated workflows	`project/.claude/skills/`
☐	Add a validation hook after 2 repeated error checks	`~/.claude/settings.json`

Start small. Let the architecture grow from actual pain points, not hypothetical ones.

The best AI assistant isn't the smartest one.

It's the one that remembers.

Built iteratively with Claude Code — one lesson at a time.